

Аудит безопасности

1. XSS (Cross-Site Scripting)

Механизм защиты

При выводе любых пользовательских данных (поля форм, сообщения об ошибках, значения из базы данных) используется функция `htmlspecialchars()`. Благодаря этому все специальные HTML-символы преобразуются в безопасные сущности. Вредоносные скрипты, введённые пользователем, отображаются как обычный текст и не могут быть выполнены в браузере.

```
<input type="text" name="name" value="<?= htmlspecialchars($values['name'] ?? '') ?>">
<?php if (isset($errors['name'])): ?>
    <div class="error-msg"><?= htmlspecialchars($errors['name']) ?></div>
<?php endif; ?>
```

Рис 1. `htmlspecialchars()`.

Результат

Уязвимости XSS отсутствуют. Защита реализована повсеместно и соответствует современным стандартам безопасности.

2. Information Disclosure (Утечка информации)

Механизм защиты

Все технические ошибки (исключения PDO, ошибки валидации, сбои в работе с БД) записываются в серверный лог через `error_log()`. Пользователь получает только общее сообщение без деталей.

Учётные данные для подключения к БД и секретный ключ JWT хранятся в переменных окружения (`getenv()`) и отсутствуют в исходном коде.

В `db.php` отсутствуют значения по умолчанию – приложение требует явного задания всех переменных окружения (аналогично `jwt.php`). Это исключает риск использования дефолтных учётных данных на сервере.

```

if ($appId === false) {
    http_response_code(500);
    echo 'Internal server error. Try again later';
    exit;
}

```

Рис 2. Обработка ошибок в бд.

```

catch (PDOException $e) {
    $pdo->rollBack();
    error_log('saveToDatabase error: ' . $e->getMessage());
    return false;
}

```

Рис 3. Запись в лог.

```

$host = getenv('DB_HOST')
$dbname = getenv('DB_NAME')
$username = getenv('DB_USER')
$password = getenv('DB_PASS')

```

Рис 4. Переменные окружения.

```

$secret = getenv('JWT_SECRET');
if ($secret === false) {
    http_response_code(500);
    die('JWT_SECRET environment variable is not set');
}

```

Рис 5. Наличие jwt secret.

Результат

Риск утечки внутренней информации (структура БД, версии библиотек, пути к файлам) полностью исключён. Конфиденциальные данные защищены.

3. SQL Injection

Механизм защиты

Все запросы к базе данных строятся исключительно через подготовленные выражения (PDO prepared statements) с использованием плейсхолдеров (:name, ?). Данные пользователя никогда не конкатенируются с

SQL-кодом – они передаются отдельно и автоматически экранируются драйвером PDO. Критически важные операции (вставка заявки и связанных языков, обновление профиля, удаление) обернуты в транзакции, что гарантирует целостность данных.

```
$pdo->beginTransaction();
$stmt = $pdo->prepare("
    INSERT INTO applications (full_name, phone, email, birth_date, gender, biography, contract_accepted)
    VALUES (:full_name, :phone, :email, :birth_date, :gender, :biography, 1)
");
$stmt->execute([
    ':full_name' => $data['name'],
    ':phone' => $data['phone'],
    ':email' => $data['email'],
    ':birth_date' => $data['birthdate'],
    ':gender' => $data['gender'],
    ':biography' => $data['bio']
]);
$appId = $pdo->lastInsertId();
```

Рис. 6 Prepared statements и плейсхолдеры.

Результат

SQL-инъекции невозможны. Приложение демонстрирует высокий уровень защиты базы данных.

4. CSRF (Cross-Site Request Forgery)

Механизм защиты

Реализован паттерн Double Submit Cookie. При каждом GET-запросе генерируется криптостойкий случайный токен, который сохраняется в сессионной cookie и передается в скрытом поле формы. При POST-запросе токен из формы сравнивается с токеном из cookie с помощью функции `hash_equals()`. CSRF-защита интегрирована во все формы: создание заявки, редактирование, вход в систему и административную панель.

```
$cookieToken = getCookieValue(CSRF_COOKIE_NAME);
if ($cookieToken === null) return false;
$formToken = $_POST[CSRF_FIELD_NAME] ?? $_GET[CSRF_FIELD_NAME] ?? '';
return hash_equals($cookieToken, $formToken);
```

Рис 7. Генерация и проверка csrf токена.

```
<form action="form.php" method="POST">
  <input type="hidden" name="_csrf" value="<?= $csrfToken ?>">
```

Рис 8. Скрытое поле с csrf.

```
if (!validateCSRFToken()) {
    http_response_code(403);
    echo 'Request denied';
    exit;
```

Рис 9. Проверка csrf при запросе.

Результат

Атаки CSRF полностью предотвращены. Механизм защищает пользователей от несанкционированных действий.

5. Include и Upload уязвимости

Include

Все подключаемые файлы заданы статически через `require_once` с фиксированными путями. Динамическое подключение файлов на основе пользовательского ввода отсутствует. Локальное или удалённое включение файлов невозможно.

```
session_start();
require_once __DIR__ . '/db.php';
require_once __DIR__ . '/config.php';
require_once __DIR__ . '/cookies.php';
require_once __DIR__ . '/validation.php';
require_once __DIR__ . '/db_functions.php';
require_once __DIR__ . '/auth.php';
require_once __DIR__ . '/jwt.php';
require_once __DIR__ . '/csrf.php';
```

Рис 10. Статическое подключение файлов.

Upload

Функционал загрузки файлов не реализован ни в одной из форм. Пользователь может передавать только текстовые данные (имя, телефон, email, сообщение), которые проходят валидацию регулярными выражениями. Отсутствуют точки входа для атак через загрузку вредоносных файлов.

Результат

Уязвимости типов Include и Upload не применимы к данному приложению по архитектурным причинам.

6. Дополнительные положительные аспекты

Аутентификация через JWT реализована вручную с помощью `hash_hmac('sha256')` без сторонних библиотек. Токены имеют ограниченный срок жизни (24 часа) и подписываются секретным ключом из переменной окружения, что обеспечивает надёжную идентификацию пользователей.

REST API поддерживает все необходимые эндпоинты (создание заявки, вход, получение профиля, обновление). API корректно обрабатывает CORS и методы GET, POST, PUT, OPTIONS. Ответы содержат понятные HTTP-статусы и структурированный JSON.

Конфигурация через переменные окружения исключает попадание секретов в репозиторий и упрощает развёртывание в разных средах.

Чистая структура кода: логика разделена на небольшие специализированные файлы (`db.php`, `auth.php`, `jwt.php`, `validation.php`, `db_functions.php`, `api.php`). Это облегчает поддержку, тестирование и развитие.

Простота развёртывания: требуется только веб-сервер с PHP и MySQL, без дополнительных фреймворков. Переменные окружения и заголовки авторизации настраиваются через `.htaccess`.

Заключение

Проведённый аудит подтверждает, что веб-приложение защищено от основных классов уязвимостей: XSS, SQL-инъекций, CSRF, Information

Disclosure, Include и Upload. Реализованные механизмы безопасности соответствуют лучшим практикам.